MAULANA ABUL KALAM AZAD UNIVERSITY OF TECHNOLOGY, WEST BENGAL

Paper Code: ESC501 Software Engineering

Time Allotted: 3 Hours Full Marks :70

The Figures in the margin indicate full marks. Candidate are required to give their answers in their own words as far as practicable

Group-A (Very Short Answer Type Question)

1. Answer any ten of the following:

(i) The CMMI was developed to combine multiple inte one framework

A) Meta model

B) Business maturity models

C) Bootstrap

D) All of the mentioned above

Ans. D) All of the mentioned above

(ii) What is the use of CMMI?

A) Decreases risks in software

B) Encouraging a productive

C) Streamlines process improvement

D) All of the mentioned above

Ans. D) All of the mentioned above

(iii) Which of the following is a building block of UML?

A) Things

B) Relationships

C) Diagrams

D) All of the mentioned

Ans. D) All of the mentioned

iv) Amongst which of the following is/are the Verification and validation activ

A) Technical reviews, quality and configuration audits

B) Algorithm analysis, development testing, usability testing

C) Qualification testing, acceptance testing, and installation testing

D) All of the mentioned above

Ans. A) Technical reviews, quality and configuration audits

(v) To achieve good design, modules should have

A). Low coupling, low cohesion

B). Low coupling, high cohesion

C). High coupling, low cohesion

D). High coupling, high cohesion

Ans. B). Low coupling, high cohesion

vi) The planning task is estimation of the resources required to accomplish the software development effort

A) True

B) False

Ans. B) False

vii) Which of the following term is best defined by the statement a structural relationship that specifies that objects of one thing are connected to objects of another?

A) Association

B) Aggregation

C) Realization

D) Generalization

Ans. A) Association

viii) A typical configuration management (CM) operational scenario involves a who is in charge of a software group

A) Project manager

B) System engineer

C) System administrator

D) All of the mentioned above

Ans. A) Project Manager

(ix) CASE Tool is

A) Computer Aided Software Engineering

B) Component Aided Software Engineering

C). Constructive Aided Software Engineering

D). Computer Analysis Software Engineering

Ans. A) Computer Aided Software Engineering

(X) All critical path activities have slack time of

A) 0

B) 1

C) 2

D) None of these

Ans. A) 0

xi) The SCM repository is the set of……..

A) Project database

B) Mechanisms and data structures

C) A tracking and control

D) None of the mentioned above

Ans. B) Mechanisms and data structures

Group-B (Short Answer Type Question)

Answer any three of the following

2. Write the short notes on: Rayleigh curve.

Ans. The Rayleigh curve, also known as the Rayleigh distribution, is a probability distribution widely used in various fields, including engineering, physics, and telecommunications, to model the magnitude of a vector with Gaussian components. It is named after Lord Rayleigh, who introduced it in the late 19th century

3. Discuss the basic COCOMO model for software cost estimation.

Ans. The basic COCOMO (Constructive Cost Model) is a widely used model for estimating the effort and cost of software development. It was developed by Barry Boehm in the 1980s and has since been revised and extended. The basic COCOMO model is based on the following key concepts:

Three Models: COCOMO is divided into three different models based on the size and complexity of the project. These are:

Basic COCOMO: Suitable for projects with less than 2,000 lines of code and simple requirements.

Intermediate COCOMO: Suitable for medium-sized projects with 2,000 to 100,000 lines of code and moderate requirements.

Detailed COCOMO: Suitable for large projects with more than 100,000 lines of code and complex requirements.

Effort Estimation: COCOMO estimates the effort required for a project based on the size of the software product, which is measured in lines of code (LOC).

Cost Estimation: COCOMO estimates the cost of a project based on the effort required and the cost of human resources.

Factors: COCOMO considers various factors that can influence the effort and cost of a project, such as the complexity of the software, the experience of the development team, and the quality of the development environment.

Mathematical Formulas: COCOMO uses mathematical formulas to estimate effort and cost based on the size of the project and the specific project characteristics.

4. Write short notes on: Software project plan.

Ans. A software project plan is a comprehensive document that outlines the scope, goals, schedule, resources, and risks associated with a software development project. It serves as a roadmap for the project team and stakeholders, providing guidance on how the project will be executed, monitored, and controlled. Here are some key points about a software project plan:

Scope: The project plan defines the scope of the project, including the features and functionality that will be delivered. It outlines the boundaries of the project and helps prevent scope creep.

Goals: The plan establishes the goals and objectives of the project, including the desired outcomes and benefits. It provides a clear direction for the project team and helps align their efforts towards achieving these goals.

Schedule: The project plan includes a detailed schedule that outlines the tasks, milestones, and deadlines for the project. It helps in tracking progress and ensuring that the project stays on track.

Resources: The plan identifies the resources required for the project, including human resources, equipment, and software tools. It helps in resource allocation and management.

Risks: The plan identifies potential risks that could affect the project and outlines strategies for mitigating these risks. It helps in proactively managing risks and minimizing their impact on the project.

Communication: The plan establishes a communication plan that outlines how information will be shared among project team members, stakeholders, and other relevant parties. It helps in ensuring that everyone is kept informed and involved in the project.

Monitoring and Control: The plan defines how the project will be monitored and controlled, including the metrics that will be used to track progress and the

procedures for making changes to the project plan. It helps in ensuring that the project stays on track and that any deviations from the plan are addressed promptly.

5. Write the short notes Re-engineering legacy systems.

Ans. Re-engineering legacy systems involves the process of modernizing and updating existing software systems to improve their functionality, maintainability, and performance. Here are some key points about re-engineering legacy systems:

Understanding the Legacy System: The first step in re-engineering a legacy system is to understand its current architecture, functionality, and limitations. This involves reviewing the codebase, documentation, and any available user feedback.

Identifying Areas for Improvement: Once the legacy system is understood, the next step is to identify areas that need improvement. This may include outdated technology, inefficient algorithms, or poor system design.

Developing a Re-engineering Strategy: Based on the identified areas for improvement, a re-engineering strategy is developed. This may involve replacing outdated technology with modern alternatives, refactoring code to improve performance, or redesigning the system architecture.

Incremental Approach: Re-engineering a legacy system is often done incrementally, with small, manageable changes made over time. This helps reduce the risk of introducing new issues and allows for a more gradual transition to the updated system.

Testing and Validation: Throughout the re-engineering process, thorough testing and validation are essential to ensure that the updated system meets the required functionality and performance standards.

User Involvement: It is important to involve users in the re-engineering process to gather feedback, address usability issues, and ensure that the updated system meets their needs.

Maintenance and Support: After the re-engineering process is complete, ongoing maintenance and support are necessary to ensure that the system continues to meet the evolving needs of the organization.

6. Write the short notes white box testing

Ans. White-box testing, also known as clear-box testing, glass-box testing, or structural testing, is a testing technique that examines the internal structure of the software being tested. Here are some key points about white-box testing:

Focus: White-box testing focuses on testing the internal logic, code structure, and flow of the software application. It is used to ensure that all code paths are executed and that the code behaves as expected.

Techniques: White-box testing techniques include statement coverage, branch coverage, path coverage, and condition coverage. These techniques help ensure that all parts of the code are tested.

Advantages: White-box testing can uncover errors in the code that may not be detected through other testing techniques. It can also help improve the code quality by identifying areas that need optimization or refactoring.

Disadvantages: White-box testing requires detailed knowledge of the internal workings of the software, which can make it more time-consuming and complex than other testing techniques. It can also be limited in its ability to detect certain types of errors, such as those related to user interface or integration with external systems.

Tools: There are several tools available for white-box testing, such as code coverage tools, static analysis tools, and debugging tools. These tools help automate the testing process and make it easier to identify and fix issues in the code.

Integration: White-box testing is often integrated into the software development process, with developers conducting unit tests to ensure that their code meets the specified requirements and standards.

Group-C (Long Answer Type Question)

Answer any three of the following

7. a) Explain the software life cycle model that incorporates risk factor.

Ans. One software life cycle model that incorporates risk factors is the Risk-Driven Model. This model recognizes that risks are inherent in software development and seeks to manage these risks throughout the project life cycle. Here's how the Risk-Driven Model works:

Risk Identification: The first step in the Risk-Driven Model is to identify potential risks that could affect the project. This involves analysing the project requirements, technology, team expertise, and external factors that could impact the project.

Risk Analysis: Once risks are identified, they are analysed to assess their likelihood and potential impact on the project. Risks are prioritized based on their severity and the level of impact they could have on the project.

Risk Mitigation: After analysing risks, strategies are developed to mitigate or reduce the impact of these risks. This may involve implementing preventive measures, such as changing the project plan or allocating additional resources, to minimize the likelihood of risks occurring.

Risk Monitoring: Throughout the project life cycle, risks are monitored to track their status and ensure that mitigation strategies are effective. New risks may also be identified as the project progresses, and these are added to the risk management plan.

Risk Response: If a risk does occur, a predefined response plan is executed to minimize its impact on the project. This may involve reallocating resources, changing the project timeline, or implementing contingency plans.

Iterative Process: The Risk-Driven Model is an iterative process, with risks being continuously identified, analysed, and managed throughout the project life cycle. This allows the project team to adapt to changing circumstances and ensure that risks are effectively managed.

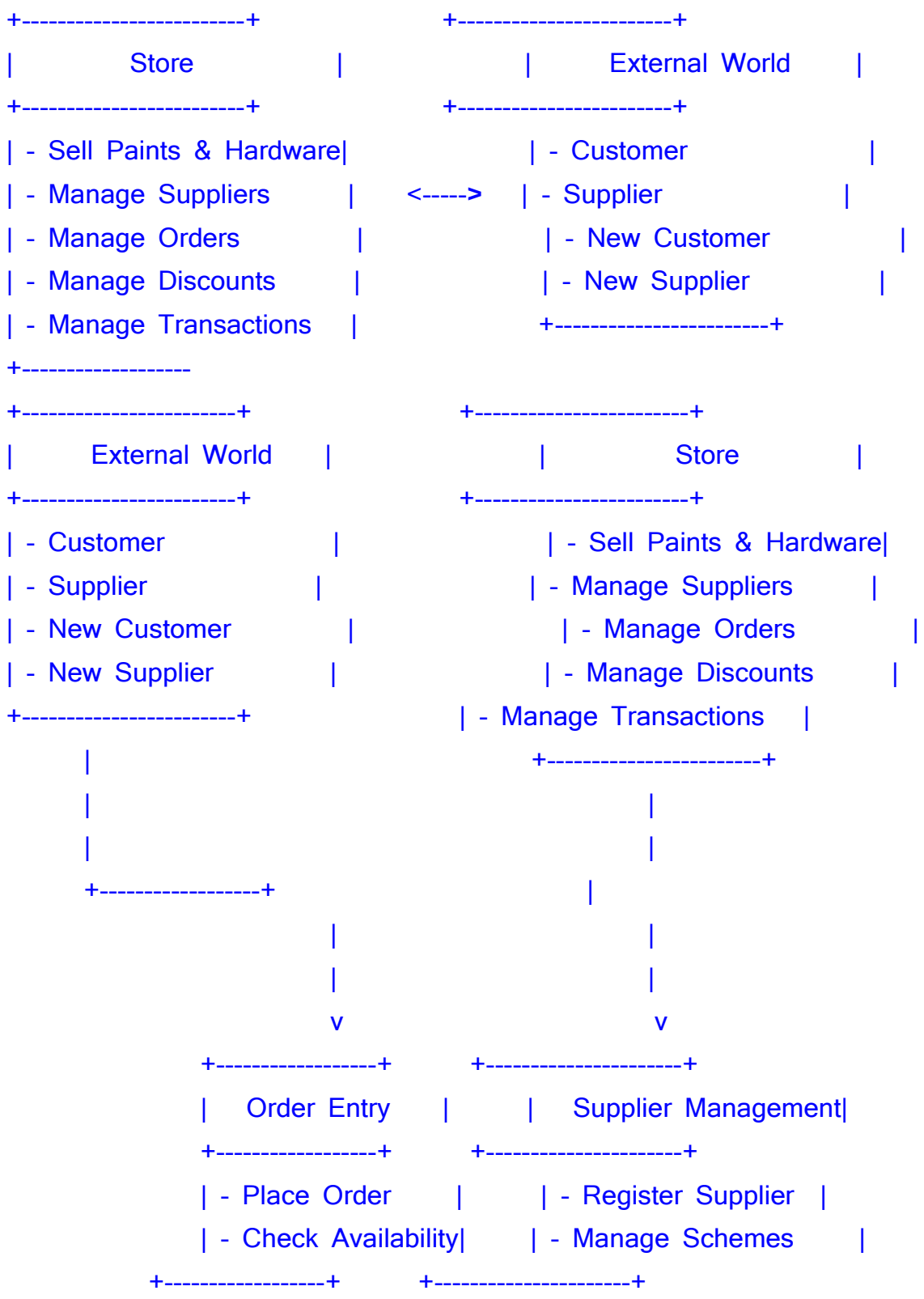
the Risk-Driven Model helps ensure that risks are identified and managed proactively throughout the software development life cycle, leading to a more successful and predictable project outcome.

b) Draw the Context level DFD and Level 1 Data Flow Diagram for the system whose requirements are summarized as follows-

A store is in the business of selling paints and hardware items. A number of reputed companies supply items to the store. New suppliers can also register with the store after providing necessary details. The customer can place the order with the shop telephonically or personally. In case items are not available, customers are informed. The detail of every new customer is stored in the company's database for future reference. Regular customers are offered discounts. Additionally details of daily transactions are also maintained. The suppliers from time to time also come up with attractive scheme for the dealers. In case, scheme is attractive for a particular item, the store places order with the company. Details of past schemes are also

maintained by the store. The details of each item i.e. item code, quantity available etc. are also maintained.

Ans. Creating a Context Level DFD and Level 1 DFD for the given system would require a more detailed understanding of the system's processes, data flows, and entities. However, based on the provided requirements, we can outline a high-level Context Level DFD and Level 1 DFD as follows



8. a) How function point analysis methodology is applied in estimation of software size? Explain. Why FPA methodology is better than LOC methodology?

Ans. Function Point Analysis (FPA) is a method used to estimate the size of a software project based on the functionality provided by the software. It is a technique that quantifies the functions provided by a software application in terms of the number and complexity of the functions. FPA is applied in the following steps:

Identify Functionality: The first step in FPA is to identify the different types of functionality provided by the software. This includes inputs, outputs, inquiries, internal logical files, and external interface files.

Assign Complexity: Each identified function is then classified based on its complexity. Complexity factors include the number of inputs and outputs, the number of files accessed, and the complexity of the processing logic.

Calculate Unadjusted Function Points: Once the functions and their complexities are identified, the unadjusted function points (UFP) are calculated. This is done by assigning weights to each function type based on its complexity and summing up the weighted function counts.

Apply Adjustment Factors: Adjustment factors are then applied to the UFP to account for various factors such as the complexity of the data, the complexity of the environment, and the experience of the development team.

Calculate Adjusted Function Points: The adjusted function points (AFP) are calculated by multiplying the UFP by the adjustment factor.

Estimate Effort and Cost: Finally, the size estimate (in function points) is used to estimate the effort and cost required to develop the software using historical productivity data.

FPA methodology is considered better than Lines of Code (LOC) methodology for software size estimation for several reasons:

Focus on Functionality: FPA focuses on the functionality provided by the software rather than the implementation details. This makes it more suitable for estimating the size of a software project based on its requirements.

Technology Independence: FPA is technology-independent, meaning that it can be applied to projects using different programming languages and technologies. This makes it more flexible than LOC, which is influenced by the programming language and coding practices.

Better Reflects Complexity: FPA takes into account the complexity of the functions provided by the software, which can vary significantly even for projects with similar LOC. This makes it a more accurate measure of software size.

Suitability for Estimation: FPA is more suitable for early estimation of software size based on high-level requirements, whereas LOC is more suitable for estimating size based on detailed design or code.

FPA provides a more comprehensive and accurate way to estimate the size of a software project compared to LOC, making it a preferred methodology for many software development projects.

b) An application has the following:10 low external inputs, 12 high external outputs, 20 low internal logical files, 15 high external interface files, 12 average external inquiries and a value adjustment factor of 1.10. What is the unadjusted and adjusted function point count?

Ans. To calculate the unadjusted function point count (UFP), we need to use the following weights for each type of function:

Low External Inputs (LEI): 3

High External Inputs (HEI): 6

Low External Outputs (LEO): 4

High External Outputs (HEO): 5

Low Internal Logical Files (LILF): 7

High Internal Logical Files (HILF): 10

Low External Interface Files (LEIF): 5

High External Interface Files (HEIF): 7

Low External Inquiries (LEq): 3

High External Inquiries (HEq): 4

The unadjusted function point count (UFP) is calculated as follows:

$=UFP=(10\times3)+(12\times5)+(20\times7)+(15\times7)+(12\times3)=30+60+140+105+36=371$

UFP=(10×3)+(12×5)+(20×7)+(15×7)+(12×3)=30+60+140+105+36=371

To calculate the adjusted function point count (AFP), we use the value adjustment factor (VAF), which is 1.10 in this case:

=AFP=UFP×VAF=371×1.10=408.1

function points are typically rounded to the nearest whole number, the adjusted function point count is 408

9. a) Define coupling and cohesion. What are the different types of coupling possible between various modules of a software system.

Ans. **Coupling:** Coupling is a measure of the degree of interdependence between modules or components in a software system. It describes how much one module relies on another. Low coupling is desirable as it indicates that modules are independent and changes in one module are less likely to impact other modules.

Cohesion: Cohesion is a measure of the degree to which the elements of a module or component are related to each other. It describes how well the responsibilities of a module are focused and unified. High cohesion is desirable as it indicates that a module has a single, well-defined purpose.

Types of Coupling:

Data Coupling: Modules communicate by passing data, but do not share data structures. This is the weakest form of coupling.

Stamp Coupling: Modules share a complex data structure, but only use part of it. This is slightly stronger than data coupling.

Control Coupling: Modules share information through control flags or variables. One module controls the behaviour of another.

External Coupling: Modules share an externally imposed format, communication protocol, or interface.

Common Coupling: Modules share global data. Changes to global data can impact multiple modules.

Content Coupling: Modules share internal data or control information. This is the strongest form of coupling and should be avoided if possible. Low coupling and high cohesion in software design, as it leads to more modular, maintainable, and flexible systems.

b) Discuss why "low coupling and high cohesion are features of good design.

Ans. Low coupling and high cohesion are considered features of good design in software engineering for several reasons:

Modularity: Low coupling and high cohesion promote modularity, which is the principle of breaking a system into smaller, independent modules. This makes the system easier to understand, maintain, and modify.

Ease of Maintenance: When modules are loosely coupled, changes to one module are less likely to have a ripple effect on other modules. This reduces the risk of introducing bugs and makes maintenance easier and less error-prone.

Flexibility and Reusability: Modules that are loosely coupled can be easily reused in other parts of the system or in other projects. High cohesion ensures that a module has a single, well-defined purpose, making it more likely to be reusable in different contexts.

Testability: Low coupling and high cohesion make it easier to test individual modules in isolation, which can improve the overall quality of the software and reduce the time and effort required for testing.

Scalability: Systems with low coupling and high cohesion are easier to scale, as new features can be added or existing features modified without significantly impacting other parts of the system.

Overall, low coupling and high cohesion lead to software that is easier to understand, maintain, and extend, making them essential features of good design in software engineering

c) Compute function point value for a project with the following domain characteristics:

No. of 10 = 30

No. of CP = 62

No. of user Inquiries - 24

No. of flps =8

No. of external interfaces = 2 \

Assume that all the complexity adjustment values are average.

Ans. To compute the function point value for the project, we will use the following weights for each type of function:

External Inputs (EI): 4

External Outputs (EO): 5

External Inquiries (EQ): 4

Internal Logical Files (ILF): 7

External Interface Files (EIF): 5

Since all complexity adjustment values are average, the complexity adjustment factor (CAF) is 1.00

The unadjusted function point count (UFP) is calculated as follows:

$UFP = (30 \times 4) + (62 \times 5) + (24 \times 4) + (8 \times 7) + (2 \times 5) = 120 + 310 + 96 + 56 + 10 = 592$

The adjusted function point count (AFP) is calculated by multiplying the UFP by the CAF:

$AFP = UFP \times CAF = 592 \times 1.00 = 59$

10. What is regression testing?

Ans. Regression testing is a type of software testing that is performed to ensure that changes or enhancements to a software application have not adversely affected existing functionality. It involves re-running previously executed test cases on the modified software to verify that the existing features still work as expected.

The primary goal of regression testing is to catch defects that may have been introduced by the changes made to the software, either intentionally (such as adding new features) or unintentionally (such as fixing bugs). Regression testing helps ensure that the overall quality of the software is maintained and that new changes do not cause unintended side effects or break existing functionality.

Regression testing can be performed manually, where testers re-run test cases manually, or it can be automated using testing tools. Automated regression testing is often preferred for large and complex software applications, as it can help save time and effort compared to manual testing.

Overall, regression testing is an important part of the software development lifecycle, as it helps ensure that software changes do not introduce new defects and that the software continues to meet its requirements and quality standards

What is alpha testing?

Ans. Alpha testing is a type of software testing performed by the internal development team or a dedicated quality assurance team within the organization. It is conducted before the software is released to external users or customers. The main purpose of alpha testing is to identify and fix bugs, issues, and usability problems in the software before it is considered ready for release.

Alpha testing is typically done in a controlled environment, such as a testing lab, and involves using the software in a simulated or real-world setting. Testers may use a variety of techniques, including functional testing, usability testing, and performance testing, to evaluate the software from different perspectives.

Alpha testing is important because it helps ensure that the software meets the organization's quality standards and is ready for broader testing with beta testers or external users. It also provides valuable feedback to the development team, allowing them to make improvements and enhancements to the software before it is released to the public

What is BETA testing?

Ans. Beta testing is a type of software testing conducted by a group of real users or customers who use the software in a real-world environment before its official release. The main goal of beta testing is to gather feedback from users about the software's functionality, usability, performance, and reliability.

Beta testing is typically conducted after alpha testing, where the software has been tested internally by the development team. Beta testing allows the software developers to get feedback from a diverse group of users who may use the software in ways that the developers did not anticipate.

There are two main types of beta testing:

Open Beta Testing: In open beta testing, the software is made available to the public, and anyone who is interested can participate in the testing. This allows for a large and diverse group of users to provide feedback on the software.

Closed Beta Testing: In closed beta testing, the software is made available to a select group of users who are chosen by the software developers. This allows for more controlled testing and allows the developers to gather feedback from specific user groups, such as existing customers or users with specific needs.

Beta testing is an important part of the software development process, as it helps identify and fix issues and improve the software's overall quality before its official release.

11. 'Software doesn't wear out' justify.

Ans. The statement "software doesn't wear out" refers to the fact that software, unlike physical objects, does not degrade over time with normal use. Instead, software tends to remain functional unless it is actively modified or affected by external factors.

There are several reasons why software is considered to not wear out:

Digital Nature: Software is essentially a set of instructions that are executed by a computer. These instructions do not degrade over time and remain intact unless intentionally modified.

No Mechanical Parts: Unlike physical objects, software does not contain any mechanical parts that can wear out or break down over time. This makes software inherently more durable.

Can be Easily Reproduced: Software can be easily copied and reproduced without any loss of quality. This means that even if a copy of the software becomes corrupted or damaged, it can be replaced with an identical copy.

Maintenance and Updates: While software itself does not wear out, it may require maintenance and updates to remain compatible with new hardware or software environments. However, these updates are typically related to changes in technology or user requirements, rather than the software itself wearing out.The statement "software doesn't wear out" reflects the fact that software is fundamentally different from physical objects in terms of its durability and longevity.

Write the IEEE definition of software engineering.

Ans. The IEEE (Institute of Electrical and Electronics Engineers) defines software engineering as:

"The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software."

Mention the characteristics of software contrasting it with characteristics of hardware.

Ans. Characteristics of Software:

Intangible: Software is intangible and cannot be touched or felt physically. It exists as a set of instructions and data that are executed by a computer.

Flexible: Software can be easily modified and updated to meet changing requirements or to fix bugs. Changes to software can be made without altering the physical components of the system.

Complexity: Software can be highly complex, with millions of lines of code and intricate interactions between different components. Managing this complexity is a key challenge in software development.

Cost of Change: Changes to software can be relatively inexpensive compared to changes in hardware. However, late changes in software can be costly and time-consuming.

Ease of Reproduction: Software can be easily reproduced and distributed, making it possible to create multiple copies of a software product at a low cost.

Characteristics of Hardware:

Tangible: Hardware is tangible and consists of physical components that can be seen and touched. Examples include processors, memory modules, and storage devices.

Less Flexible: Hardware is less flexible than software and is more difficult and expensive to modify. Changes to hardware often require physical alterations to the system.

Physical Limits: Hardware is subject to physical limits such as size, weight, and power consumption. These limits can constrain the design and functionality of hardware systems.

Cost of Change: Changes to hardware can be expensive and time-consuming. Designing and manufacturing new hardware components can require significant investment

Difficult to Reproduce: Hardware is difficult to reproduce and distribute compared to software. Manufacturing hardware components requires specialized equipment and expertise.

Software is intangible, flexible, and easier to reproduce and modify compared to hardware. Hardware, on the other hand, is tangible, less flexible, and subject to physical limits and higher costs of change.